BRANDME

App Development Plan

A staged development roadmap for building Toronto's creator discovery platform

React Native | Expo SDK 54 | Supabase | Mapbox | PostHog

March 2026 · Updated Build Plan · Confidential

DOCUMENT OVERVIEW

Overview

BrandMe is a mobile-first social discovery platform for Toronto's creator community. It connects content creators, brand owners, artists, and anyone building an online presence with one another and with local events — think LinkedIn for creatives, built specifically for Toronto.

This document reflects the updated build plan after the Stage 6 refactor. The posts feed has been removed to keep the product focused on discovery and events. Portfolio images remain as a showcase feature on profiles. The Create tab has been renamed **Host** to better reflect its singular purpose: creating events.

REFACTOR SUMMARY

What Changed

Removed	Posts feed on Home screen — Home is now a pure events feed
Removed	Post creation form on the Host tab — Host tab is events-only
Removed	posts table references from app code (table may remain dormant in DB)
Renamed	Create tab → Host tab (icon and label updated throughout)
Kept	portfolio_images table and portfolio grid on profiles
Kept	Supabase Storage bucket — still used for profile pictures, portfolio images, and event covers
Kept	follows / follower counts — still core to creator discovery

GUIDING PRINCIPLES

Principles

Mock Data First, Real Data Later

All screens use typed mock data until the UI is stable. TypeScript interfaces mirror the DB schema so swapping in real Supabase calls is seamless. Authentication is the sole exception — set up with Supabase from the start.

Direct Supabase Calls from the Frontend

Standard CRUD (profiles, events, RSVPs, follows) goes through @supabase/supabase-js. RLS policies handle authorization at the DB level. FastAPI is introduced only for complex geospatial queries or sensitive business logic.

One Stage, One Testable Milestone

Each stage ends with a clear checkpoint you can tap through on your phone. No stage depends on the next being complete.

Host Tab = Events Only

The Host tab has one job: creating an event. There is no 'what do you want to create?' decision point. This keeps the interaction model simple and intentional.

CONTENTS

Table of Contents

Stage 0	Project Scaffolding & TypeScript Foundation
Stage 1	Authentication & Onboarding
Stage 2	Account Page
Stage 3	Creator Map
Stage 4	Events System
Stage 5	My Events & Event Management
Stage 6	Social Graph & Portfolio ✥ Updated
Stage 7	Go Live — Replace Mock Data with Supabase
Stage 8	Polish, Search & Discovery
Stage 9	Analytics, Testing & Launch Prep
Appendix	Database Schema Reference

STAGE 0

Project Scaffolding & TypeScript Foundation

Creates the project skeleton and establishes conventions every future stage will follow. By the end you have a running Expo app with tab navigation and a complete type system that mirrors the database schema.

TASKS

#	Task	Details
1	**Initialize Expo project**	Run create-expo-app with the blank TypeScript template. Confirm the app runs on your physical device via Expo Go by scanning the QR code.
2	**Install core dependencies**	React Navigation (bottom tabs + native stack), TanStack Query, @supabase/supabase-js, react-native-maps. Do not configure Supabase yet — just install the package.
3	**Build bottom tab navigator**	Five tabs: Home, Map, Host, My Events, Account. Each tab renders a placeholder screen showing the tab name. Use icons from @expo/vector-icons. The Host tab uses a distinct icon (e.g. calendar-plus) to signal its purpose.
4	**Create /types directory**	Define TypeScript interfaces for all entities: User, Profile ('CREATOR' | 'BUSINESS'), Niche, ExternalLink, Event, EventRSVP, Follow, PortfolioImage. Mirror the DB schema exactly. No Post type.
5	**Create /mock directory**	Build mock data files: 8–10 sample profiles (mix of creators and businesses), 5–6 events with varied types and dates, a master niche list of 15–20 entries. Export as typed arrays.
6	**Set up project structure**	Folder conventions: /src/screens, /src/components, /src/navigation, /src/types, /src/mock, /src/hooks, /src/utils, /src/constants. Add a global theme/colors file.

✓ END-OF-STAGE CHECKPOINT

- App launches on your phone via Expo Go
- Five tabs visible and tappable — Home, Map, Host, My Events, Account
- Each tab shows its placeholder screen with correct title
- No Post type exists in the type system
- All TypeScript types compile without errors
- Project folder structure is established and consistent

STAGE 1

Authentication & Onboarding

The only stage where real Supabase infrastructure is set up before the UI is complete. Authentication is foundational — navigation guards, profile ownership, and RLS all depend on knowing who the current user is.

TASKS

#	Task	Details
1	**Create Supabase project**	Set up a new Supabase project. Enable email/password auth. Store the project URL and anon key in an environment config file (not hardcoded).
2	**Create database tables**	In the Supabase SQL editor, create the profiles, niches, profile_niches, and external_links tables. Seed the niches table with 15–20 categories. Enable the PostGIS extension.
3	**Configure Supabase client**	Initialize the client in /src/lib/supabase.ts using AsyncStorage for session persistence. Wrap the app in a TanStack QueryClientProvider.
4	**Build auth screens**	Welcome/Landing screen, Sign Up screen (email + password), and Log In screen. Use Supabase auth.signUp() and auth.signInWithPassword(). Handle loading states and error messages.
5	**Build onboarding flow**	Multi-step onboarding after sign-up: (1) Choose Creator or Business. (2) Display name and bio. (3) Select niches (multi-select chips). (4) Upload profile picture via Expo ImagePicker. (5) Set location (device coordinates or manual entry). Write all data to profiles and profile_niches on completion.
6	**Add navigation guards**	Not logged in → auth stack. Logged in but onboarding incomplete → onboarding stack. Fully onboarded → main tab navigator.

✓ END-OF-STAGE CHECKPOINT

- New user can sign up and complete the full onboarding flow
- Profile data (type, name, bio, niches, picture, location) is written to Supabase
- Returning users can log in and are taken directly to the main app
- Logging out returns the user to the Welcome screen
- Navigation guards prevent unauthenticated access

STAGE 2

Account Page

Built early because it forces a complete read/write loop with Supabase and produces a reusable ProfileCard component used across the map, events, and social features. At this stage the Account page reads from real Supabase data; everything else still uses mock data.

TASKS

#	Task	Details
1	**Build profile view screen**	Display current user's profile: profile picture, display name, bio, profile type badge, years in business, niche tags, external links (tappable icons), follower/following counts (hardcoded 0 for now).
2	**Build profile edit screen**	Edit mode for: display name, bio, years in business, profile picture, location, niches, and external links. Save changes to Supabase on submit.
3	**Create reusable ProfileCard**	Compact card (picture, name, niche tags, profile type) for use in search results, map bottom sheets, and follower lists.
4	**Create reusable NicheTag**	Styled chip/badge for niche labels. Used on profiles, filters, and event cards.
5	**Add Log Out button**	Calls Supabase auth.signOut() and returns the user to the Welcome screen.
6	**Build View Other Profile screen**	Read-only profile view. Accepts a profile ID as a navigation param, fetches from Supabase. Includes a Follow/Unfollow button (UI only for now).

✓ END-OF-STAGE CHECKPOINT

- Your own profile displays all data from onboarding
- Every field can be edited and changes persist after reopening
- Tapping external links opens the correct URL
- ProfileCard and NicheTag components render correctly
- Log out works and returns to Welcome screen
- View Other Profile renders when given a profile ID

STAGE 3

Creator Map

The app's signature feature. Users discover creators, businesses, and events near them. This stage focuses on Creator Mode using mock profile data. Event Mode is populated in Stage 5. The map is wired to Supabase in Stage 7.

TASKS

#	Task	Details
1	**Set up react-native-maps with Mapbox**	Install and configure react-native-maps. Obtain a Mapbox API key and configure a custom dark/branded map style. Center on Toronto by default.
2	**Render creator pins from mock data**	Plot mock profiles as custom markers. Distinct pin designs for Creators vs. Businesses. Implement marker clustering for zoomed-out views.
3	**Build bottom sheet for pin taps**	Tapping a creator pin slides up a bottom sheet with the ProfileCard. Include a 'View Full Profile' button navigating to View Other Profile.
4	**Add mode toggle (Creator / Event)**	Toggle or segmented control at the top of the map. Creator Mode shows creator/business pins. Event Mode shows placeholder pins or an empty 'Coming soon' state.
5	**Add niche filter**	Horizontally scrollable niche chip row. Selecting a niche filters visible pins. Filter state persists across mode switches.
6	**Add search bar**	Filters pins by display name. Persists across mode switches. Works in combination with niche filter.
7	**Handle location permissions**	Request on map load. If granted, show 'my location' dot. If denied, default to central Toronto view.

✓ END-OF-STAGE CHECKPOINT

- Map loads centered on Toronto with custom Mapbox style
- Mock creator and business pins are visible with distinct styling
- Tapping a pin opens a bottom sheet with the ProfileCard
- Niche filter chips filter visible pins correctly
- Search bar filters pins by name
- Creator/Event mode toggle is present (Event mode is empty for now)
- Current location dot appears if permissions are granted

STAGE 4

Events System

Builds the complete events lifecycle: creating events, browsing them in a feed, RSVPing, and seeing them on the map. The Create Event flow writes to Supabase; the feed uses mock data until Stage 7.

TASKS

#	Task	Details
1	**Create events & event_rsvps tables**	events table: id, host_id, title, description, event_type enum (FASHION_SHOW | NETWORKING | WORKSHOP | LISTENING_PARTY | COMMUNITY | OTHER), start_time, end_time, latitude, longitude, address_name, status (DRAFT | PUBLISHED | CANCELLED), cover_image_url, created_at. event_rsvps table: event_id, profile_id, status (INTERESTED | REQUESTED | GOING | DECLINED).
2	**Build Create Event screen (Host tab)**	Form accessible from the Host tab: title, description, event type picker, date/time pickers, privacy toggle (Public / Exclusive), optional cover image upload via Expo ImagePicker (stored in Supabase Storage), location picker (mini-map tap or address search). Writes to Supabase on submit.
3	**Build Home screen (Events feed)**	Vertically scrollable feed of upcoming events using mock data. Each card shows: cover image (if present), title, event type badge, privacy indicator, host name and profile picture, date/time, location name, Interested/Going counts.
4	**Build Event Detail screen**	Full event view: all event info, cover image, mini-map, host ProfileCard, lists of Interested/Going attendees (mock), and RSVP action buttons.
5	**Implement RSVP buttons**	Interested and Going buttons on cards and detail screen. Toggle state stored locally for now (wired to Supabase in Stage 7). Exclusive events show a 'Request to Attend' flow instead of direct Going.
6	**Add niche filter to feed**	Same horizontally scrollable niche chip row used on the map. Filters events based on host's niches or an event's associated niche.
7	**Populate Event Mode on the map**	Event Mode shows mock events as pins (calendar icon style). Tapping opens a bottom sheet with event summary and 'View Details' button.

✓ END-OF-STAGE CHECKPOINT

- Create Event form is accessible via the Host tab
- New event is written to Supabase with cover image stored correctly
- Home screen shows a scrollable feed of mock events

- Tapping an event card opens the full Event Detail screen
- RSVP buttons toggle visually and store state locally
- Exclusive event flow shows Request to Attend instead of Going
- Niche filter works on the events feed
- Event Mode on the map shows mock event pins with tappable bottom sheets

STAGE 5

My Events & Event Management

Completes the events experience from the user's perspective — viewing RSVPs, managing hosted events, and handling cancellations.

TASKS

#	Task	Details
1	**Build My Events screen**	Two tabs or segmented control: Interested and Going. Each shows event cards filtered by RSVP status. Pull from local RSVP state at this stage.
2	**Add 'My Hosted Events' section**	On the Account page, add a section showing events the current user has created. Each card shows title, date, privacy badge, and RSVP counts. Tapping opens Event Detail.
3	**Build event edit functionality**	Host can edit title, description, time, location, privacy, and cover image for their own events. Include a 'Cancel Event' option that sets status to CANCELLED.
4	**Add empty states**	Friendly empty state messages for Interested, Going, and Hosted tabs when no events exist (e.g. 'No events yet — explore the map to find what's happening!').

✓ END-OF-STAGE CHECKPOINT

- My Events screen shows separate Interested and Going lists
- Account page shows events you've created with RSVP counts and privacy badges
- You can edit your own events and see changes reflected
- Cancelling an event marks it as cancelled in the feed
- Empty states display correctly in all tabs

STAGE 6

Social Graph & Portfolio

Adds the social layer (following, follower lists) and the portfolio showcase feature on profiles. The posts feed has been removed — this stage no longer includes post creation or a posts feed. Supabase Storage is used for profile pictures and portfolio images only.

Note: The posts table may remain dormant in Supabase — no app code references it. The portfolio_images table and Supabase Storage bucket are fully active.

TASKS

#	Task	Details
1	**Create follows table**	follows table: follower_id (FK to profiles.id), following_id (FK to profiles.id), created_at. Composite PK on both IDs. Add follower_count and following_count as denormalized columns on profiles. Add a DB trigger to increment/decrement these counts on insert/delete.
2	**Implement follow/unfollow**	Follow/Unfollow button on View Other Profile screen. Tapping writes to follows table and updates both users' denormalized counts. Button state reflects the current relationship.
3	**Build follower/following lists**	Tapping follower or following count on the Account page opens a list of ProfileCards. Each card is tappable to navigate to that user's profile.
4	**Set up Supabase Storage**	Create/confirm storage bucket for user media. Configure access policies: users can upload to their own folder; all authenticated users can read. Used for profile pictures and portfolio images.
5	**Build portfolio upload flow**	On the Account page, allow the profile owner to upload portfolio images from their camera roll (Expo ImagePicker). Images are stored in Supabase Storage, and a portfolio_images record is created with a display_order. Support reordering and deletion.
6	**Add portfolio grid to profiles**	On both the Account page and View Other Profile screen, render a grid of portfolio images below the bio. Tapping an image opens a full-screen lightbox view.

✓ END-OF-STAGE CHECKPOINT

- You can follow and unfollow other users from their profile
- Follower and following counts update correctly on profiles
- Tapping counts shows a list of ProfileCards
- Portfolio images can be uploaded, reordered, and deleted from your own profile

- Portfolio grid is visible on both your profile and other users' profiles
- No posts feed or post creation exists anywhere in the app

STAGE 7

Go Live — Replace Mock Data with Supabase

The UI is now stable. This stage is a dedicated wiring pass replacing all mock data with real Supabase queries. Because TypeScript types already mirror the schema, this is methodical rather than a rewrite.

TASKS

#	Task	Details
1	**Wire up the Creator Map**	Replace mock profile pins with a Supabase PostGIS query (ST_DWithin via .rpc()). Fetch profiles within the visible map bounds only.
2	**Wire up the Events feed**	Replace mock events with a Supabase query: upcoming events ordered by start_time, joined with host profile data. Apply niche filters as query parameters.
3	**Wire up RSVPs**	Move RSVP state to the event_rsvps table. Interested/Going buttons read from and write to Supabase. My Events screen queries event_rsvps filtered by the current user. Exclusive event request flow writes REQUESTED status.
4	**Wire up Event Mode on the map**	Event pins come from a Supabase PostGIS query filtered by map region and optionally by niche.
5	**Enable Row Level Security (RLS)**	Write RLS policies for every table. Users can read all profiles and events but only update their own. Users can only insert/delete their own RSVPs and follows. Portfolio images readable by all, writable only by the owner. Test every policy.
6	**Add TanStack Query caching**	Wrap all Supabase calls in useQuery / useMutation hooks. Configure stale times: profiles cache longer; events, RSVPs, and follows refresh more frequently.
7	**Handle loading and error states**	Loading skeletons or spinners on every data-fetching screen. Error boundaries and retry logic. Pull-to-refresh on Events feed and My Events screen.

✓ END-OF-STAGE CHECKPOINT

- Every screen reads from and writes to Supabase — no mock data remains
- The map loads real profiles and events by geographic proximity
- RSVPs and portfolio images persist in the database
- RLS policies are tested and unauthorized writes are rejected
- Loading spinners, skeletons, and error states are visible throughout
- Pull-to-refresh works on all list screens

STAGE 8

Polish, Search & Discovery

With all core features on real data, this stage focuses on quality and discoverability — going from 'it works' to 'it feels good to use.'

TASKS

#	Task	Details
1	**Build global search**	Search screen accessible from Home. Search across creators (name, niche, bio) and events (title, description, location). Results in two sections: People and Events. Uses ProfileCard and event card components.
2	**Refine map interactions**	Smooth animations on pin selection. Bottom sheet supports swipe-to-dismiss and swipe-to-expand. 'Recenter on me' button. Cluster animations when zooming in/out.
3	**Visual polish pass**	Consistent spacing, typography, and color across all screens. Smooth transitions. Sharp on both iOS and Android. Test on multiple screen sizes.
4	**Add push notifications**	Integrate Expo Notifications. Notify users when: someone follows them, an event they're interested in starts soon (24-hour reminder), or a new event is posted in a niche they follow. Register push token with Supabase.
5	**Implement image caching**	Use expo-image or react-native-fast-image for efficient loading and caching. Profile pictures, portfolio images, and event cover images load once and cache locally.
6	**Accessibility pass**	Accessibility labels on all interactive elements. Test with VoiceOver (iOS) and TalkBack (Android). Sufficient color contrast on all text.

✓ END-OF-STAGE CHECKPOINT

- Global search returns relevant creators and events quickly
- Map interactions feel smooth with animations and expandable bottom sheets
- App looks visually polished across all screens
- Push notifications delivered for follows and event reminders
- Images load fast and don't re-download on revisit
- App is usable with screen readers

STAGE 9

Analytics, Testing & Launch Prep

The final stage prepares BrandMe for real users. Instrument the app with analytics, build for the app stores, and run the first round of community testing in Toronto.

TASKS

#	Task	Details
1	**Integrate PostHog**	Install PostHog React Native SDK. Track: screen views, profile views, event RSVPs, follows, portfolio views, map interactions (zoom, pin taps, mode switches), search queries, onboarding completion rate.
2	**Set up EAS builds**	Configure eas.json for development, preview, and production build profiles. Run a preview build and install outside Expo Go to test full native experience (push notifications, maps, image picker).
3	**Create App Store assets**	Design app icon, splash screen, and App Store/Play Store screenshots. Write app description and metadata. Set up App Store Connect and Google Play Console listings.
4	**Seed real data for testing**	Create 20–30 realistic test profiles across niches and Toronto locations. Create 10–15 upcoming events with cover images. Data should feel like an active platform.
5	**Run community test**	Distribute preview build to 10–20 Toronto-based creators via TestFlight and Android internal testing. Gather feedback on: onboarding friction, map usability, event discovery, portfolio showcase, overall value. Monitor PostHog for drop-off.
6	**Bug fix and iterate**	Address critical bugs surfaced during testing. Prioritize issues blocking core actions (signing up, finding creators, RSVPing, viewing portfolios) over cosmetic polish.

✓ END-OF-STAGE CHECKPOINT

- PostHog dashboard shows tracking for all core user actions
- App runs as a standalone build on iOS and Android
- App Store assets and metadata are ready for submission
- Test data makes the app feel populated and active
- Community testers have provided feedback
- Critical bugs from testing are resolved

- App is ready for its first public release in Toronto

APPENDIX

Database Schema Reference

Complete table definitions for BrandMe, listed in creation order (respecting foreign key dependencies). The posts table is intentionally excluded from the app; it may remain dormant in Supabase if desired.

profiles

- id — UUID, PK
- user_id — FK to auth.users.id
- display_name — text
- bio — text
- profile_type — enum: 'CREATOR' | 'BUSINESS'
- years_in_business — integer
- profile_picture_url — text (Supabase Storage)
- latitude, longitude — float (PostGIS coordinates)
- follower_count, following_count — integer, default 0 (denormalized)
- created_at — timestamp

niches

- id — PK
- name — text, unique (e.g. Streetwear, Cinematography, Barbering, Music Production)

profile_niches

- profile_id — FK to profiles.id
- niche_id — FK to niches.id
- Composite PK on (profile_id, niche_id)

external_links

- id — PK
- profile_id — FK to profiles.id
- platform_name — text (e.g. Instagram, TikTok, YouTube)

• url — text

events

• id — UUID, PK

• host_id — FK to profiles.id

• title — text

• description — text

• event_type — enum: FASHION_SHOW | NETWORKING | WORKSHOP | LISTENING_PARTY | COMMUNITY | OTHER

• privacy — enum: PUBLIC | EXCLUSIVE

• start_time, end_time — timestamp

• latitude, longitude — float

• address_name — text (e.g. 'The Liberty Village Commons')

• cover_image_url — text, nullable (Supabase Storage)

• status — enum: DRAFT | PUBLISHED | CANCELLED

• created_at — timestamp

event_rsvps

• event_id — FK to events.id

• profile_id — FK to profiles.id

• status — enum: INTERESTED | REQUESTED | GOING | DECLINED

• Composite PK on (event_id, profile_id)

follows

• follower_id — FK to profiles.id

• following_id — FK to profiles.id

• created_at — timestamp

• Composite PK on (follower_id, following_id)

• DB trigger increments/decrements denormalized counts on profiles

portfolio_images

• id — PK

• profile_id — FK to profiles.id

- image_url — text (Supabase Storage)
- display_order — integer
- created_at — timestamp

Note: The posts table is not listed here. It may exist dormant in your Supabase project but is not referenced anywhere in the app codebase.